



BOARD OF INTERMEDIATE EDUCATION, KARACHI

HSC Computer Science · Science General & Humanities

Theory 75 marks + Practical 25 marks = 100 per part

COMPLETE CLASSROOM LECTURES · HSC PART-II

Computer Science XII

Option I (C) · Option II (VB) · 2019 units

WHAT THESE NOTES COVER

- ▶ Part A — Programming using C (BIEK Option I, Turbo C++ style)
- ▶ Part B — Visual Basic (BIEK Option II, event-driven board topics)
- ▶ Part C — SDLC, pointers, OOP, files, database/SQL, multimedia, wireless
- ▶ Practical journal list from the Sindh Curriculum 2019
- ▶ Model MCQs, short questions and 8-mark long questions

Contents

How this Class XII book is organised	3
Paper pattern (Computer Science II)	3
Part A — Option I: Programming using C	3
Part B — Option II: Programming using Visual Basic	9
Part C — Sindh Curriculum 2019 units	11
Part D — Grade XII practicals	18
Model practice — Option I (C)	18
Model practice — new-book long questions	20
Quick glossary (XII)	20

How this Class XII book is organised

BIEK Computer Science Paper-II is still published as two independent options: Option I Programming using C, and Option II Programming using Visual Basic. You attempt only the option your college taught. The Sindh Curriculum 2019 adds C++ OOP, pointers, file handling, SDLC, databases, multimedia, and wireless communication. This volume teaches all of that in four parts so both 'old paper' and 'new book' students can revise.

Part	Who it is for	Contents
A	Every BIEK candidate taking Option I	Complete C language lectures (board pattern)
B	Candidates taking Option II	Visual Basic 6-style board lectures
C	New-book / 2019 curriculum	SDLC, pointers, OOP C++, files, SQL, multimedia, wireless
D	Everyone	Practicals, model papers, glossary

Board exam tip

If your admission slip says Computer Science II (Opt I/II), your 75-mark theory paper is C or VB, not a mix. Still read Part C: many colleges have started the 2019 units in internal exams and practicals.

Paper pattern (Computer Science II)

Section	Type	Typical demand
A	MCQs, all compulsory	Syntax, output of snippets, operators, loops, arrays
B	Short questions, attempt 10 of 15 (recent scheme 3 marks each)	Definitions + tiny programs
C	Detailed, attempt 2 of 3 (8 marks)	Loops comparison, pattern program, if→switch, sort

Part A — Option I: Programming using C

Lecture C1 — Language, IDE and program structure

C was designed by Dennis Ritchie at Bell Labs (early 1970s) to write the UNIX operating system. It is a middle-level language: high-level control with low-level memory access. BIEK papers still expect the name Dennis Ritchie.

Definition — IDE

Integrated Development Environment: one application that provides editor, compiler, linker and debugger. Turbo C++ 3.0 (DOS) is what many Karachi colleges still demonstrate. Modern options: Dev-C++, Code::Blocks, VS Code + gcc.

Turbo C++ key	Action
F2	Save
Alt+F9	Compile
Ctrl+F9	Run (compile if needed)

Turbo C++ key	Action
Alt+F5	See user screen / output
F10	Menu bar

```

#include <stdio.h>
#include <conio.h>      /* Turbo C++ only */

void main(void)        /* board papers use this; ISO C prefers int main(void) */
{
    printf("Welcome to BIEK Computer Science II");
    getch();           /* holds the output screen in Turbo C++ */
}

```

Minimum C program in board style

- `#include` is a preprocessor directive (must start with `#`). It inserts a header file.
- `stdio.h` — standard input/output (`printf`, `scanf`). `conio.h` — console (`getch`, `clrscr`) on Turbo C, not portable.
- `main` is where execution starts. There is exactly one `main`.
- Every statement ends with a semicolon. Comments: `/* block */` or `// line (C++ / C99)`.

Common mistake

`void main(void)` is accepted in BIEK marking. In a modern compiler, write `int main(void)` and `return 0`.

Lecture C2 — Tokens, types, variables, operators

A token is the smallest meaningful piece of a program: keyword, identifier, constant, string, operator, punctuator.

Identifiers

- Start with a letter or underscore; then letters, digits, underscore.
- Valid: `record1`, `_tax`, `Marks`. Invalid: `$tax`, `name and address (space)`, `2day` (leading digit).
- Keywords (`int`, `if`, `return`, `void`, ...) cannot be identifiers. C is case-sensitive: `marks` ≠ `Marks`.

Constants

Kind	Board examples	Invalid / why
Integer	1527, 0, -8	27,822 (comma)
Float / real	0.576, 2e-8, 5.234	1,232.5 (comma)
Character	'a', '9', '\n'	"a" is a string, not a char
String	"Pak"	'Pak' is not a legal char

Basic data types (16-bit Turbo C sizes — quote these in BIEK)

Type	Typical size	Approx. range (Turbo C)
char	1 byte	-128 to 127 (signed)
int	2 bytes	-32768 to 32767
long int	4 bytes	about ±2 billion
float	4 bytes	6–7 decimal digits
double	8 bytes	15 decimal digits

Remember

On a modern 32/64-bit gcc, int is usually 4 bytes. If the paper says 'typical memory requirement', give the Turbo C column and you will match the marking scheme.

Operators

Group	Symbols	Notes
Arithmetic	+ - * / %	% needs integers; 4%7 is 4
Assignment	= += -= *= /= %=	a += 1 same as a = a+1
Increment	++ --	pre vs post changes the value used in an expression
Relational	< <= > >= == !=	Result is 1 (true) or 0 (false)
Logical	&& !	AND, OR, NOT

```

int x = 3;
int y = x++;    /* post: y gets 3, then x becomes 4 */
printf("x=%d and y=%d", x, y);    /* x=4 and y=3 */

int a = 2, b = 2;
if (a != 0)
    b = 0;
else
    b *= 10;
printf("%d", b);    /* prints 0 */

```

Post-increment and if-without-braces — both appear as MCQs

Lecture C3 — Input, output, format specifiers, escape sequences

Specifier	Meaning
%d or %i	int (decimal)
%c	char
%f	float (printf); also used for double in printf
%lf	double in scanf
%s	string (char array)
%u	unsigned
%ld	long
%o / %x	octal / hexadecimal

Escape	Effect
\n	new line
\t	horizontal tab
\\	prints a backslash
\"	prints a double quote
\0	null terminator of a string

printf writes formatted output; scanf reads formatted input (needs & on ordinary variables). gets / puts are simpler string I/O (gets is unsafe; in a modern lab use fgets). To print the two characters \n on screen: printf("\\n");

```

int n;
float avg;
char grade;
printf("Enter marks: ");
scanf("%d", &n);
avg = n / 1.0;
grade = (n >= 40) ? 'P' : 'F';
printf("Marks=%d Average=%.2f Grade=%c\n", n, avg, grade);

```

Lecture C4 — Decision making

if chooses one path. if-else chooses between two. else-if ladder chooses among many ranges. switch chooses among integer/char labels.

```

int condition;
scanf("%d", &condition);
if (condition == 1)
    printf("Red colour");
else if (condition == 2)
    printf("Black Colour");
else if (condition == 3)
    printf("White Colour");
else
    printf("No Colour");

/* equivalent switch – a frequent Section C conversion */
switch (condition) {
    case 1: printf("Red colour"); break;
    case 2: printf("Black Colour"); break;
    case 3: printf("White Colour"); break;
    default: printf("No Colour");
}

```

if-else ladder converted to switch

Point	if / else-if	switch
Condition type	Any expression (including ranges)	Integer or char equality
Ranges (marks >= 80)	Natural	Awkward (need many cases)
Fall-through	No	Yes, if you omit break
Default path	else	default

Common mistake

switch cannot test a float. Relational operators compare; logical operators combine comparisons. continue does not belong with switch (MCQ).

Lecture C5 — Loops

C has three loops: for, while, do-while. There is no repeat-until (that is Pascal). goto-label can make an unstructured loop; avoid it.

```

int i;
for (i = 1; i <= 5; i++)
    printf("%d ", i);

i = 1;
while (i <= 5) {
    printf("%d ", i);
    i++;
}

i = 1;
do {
    printf("%d ", i);
    i++;
} while (i <= 5);

```

for / while / do-while

- for is best when you know the count. while is best when you wait for a condition (e.g. sentinel). do-while is best for menus because the body runs at least once.
- Nested for: outer row, inner column — used for tables and star patterns.
- break leaves the loop immediately. continue skips the rest of this iteration.

Pattern programs (Section C)

```

/* 666666
   55555
   4444
   333
   22
   1      */
int r, c;
for (r = 6; r >= 1; r--) {
    for (c = 1; c <= r; c++)
        printf("%d", r);
    printf("\n");
}

```

Classic descending digit triangle

```

int n, r, c;
printf("Size: ");
scanf("%d", &n);
for (r = 1; r <= n; r++) {
    for (c = 1; c <= r; c++)
        printf("*");
    printf("\n");
}

```

Right triangle of stars (also a function in XI practicals)

Lecture C6 — Functions

Definition — Function

A self-contained block that performs a specific task and may return a value. C programs are collections of functions; main is only the first that runs.

```

int add(int a, int b);    /* prototype / declaration */

void main(void) {
    int s;
    s = add(3, 4);        /* call; 3 and 4 are actual arguments */
    printf("%d", s);
}

int add(int a, int b) {   /* definition; a,b formal parameters */
    return a + b;        /* return sends a value back */
}

```

Declaration, call, definition

Idea	Board wording
Prototype	Tells compiler the name, return type and arguments before the call
Pass by value	Function receives a copy; cannot change the caller's variable
Local (automatic)	Declared inside a function; born at entry, dead at exit
Global (external)	Declared outside all functions; visible from the declaration to the end of the file
Library functions	printf, scanf, sqrt, abs, strlen, getch, ...

Q. Write the syntax of function definition, calling and prototype.

A. Prototype: return_type name(type1, type2); Call: name(expr1, expr2); Definition: return_type name(type1 p1, type2 p2) { statements; return value; }

Lecture C7 — Arrays and strings

Array elements live in consecutive (sequential) memory. Valid initialisation: int x[6] = {2,3,4,12,5,4};

```

int arr[10] = {1,2,3,4,5};
printf("%d", arr[5]);    /* 0 – remaining elements are 0 if partial init */

/* sort ten numbers (selection sort) */
int a[10], i, j, t;
for (i = 0; i < 10; i++)
    scanf("%d", &a[i]);
for (i = 0; i < 9; i++)
    for (j = i + 1; j < 10; j++)
        if (a[i] > a[j]) {
            t = a[i]; a[i] = a[j]; a[j] = t;
        }
for (i = 0; i < 10; i++)
    printf("%d ", a[i]);

```

Partial init + ascending sort — both asked in model papers

Strings: char s[20] = "Karachi"; Use %s with printf/scanf (scanf stops at space; gets reads a line). strlen, strcpy, strcat, strcmp live in string.h.

Lecture C8 — Extra board programs to memorise

```

/* even / odd, max of three, factorial, Fibonacci, prime */
int n, i, f = 1, a = 0, b = 1, c, flag = 1;
scanf("%d", &n);
printf(n % 2 == 0 ? "Even\n" : "Odd\n");

/* factorial */
for (i = 1; i <= n; i++) f *= i;
printf("Fact=%d\n", f);

/* first n Fibonacci */
printf("%d %d ", a, b);
for (i = 3; i <= n; i++) { c = a + b; printf("%d ", c); a = b; b = c; }

/* prime */
if (n < 2) flag = 0;
for (i = 2; i <= n / 2; i++)
    if (n % i == 0) { flag = 0; break; }
printf(flag ? "Prime" : "Not prime");

```

```

/* swap two numbers using a function (pass by value needs pointers in C
   to truly swap – board sometimes only prints swapped copies) */
void swap_print(int x, int y) {
    int t = x; x = y; y = t;
    printf("%d %d", x, y);
}

```

Part B — Option II: Programming using Visual Basic

Visual Basic (VB 6.0 in most BIEK labs) is an event-driven language from Microsoft. You draw controls on a form; code runs when an event (Click, Load, Change) happens. Attempt this part only if your college offered Option II.

V1 — IDE and application states

Window / state	Use
Form designer	Draw labels, text boxes, buttons
Code window	Write event procedures
Properties window	Caption, Name, Font, ForeColor, Visible, Enabled
Project explorer	Forms, modules, project name
Toolbox	Pointer, Label, TextBox, CommandButton, ListBox, ComboBox, ...
Immediate window	Test expressions at design/break time (print x)
Design / run / break states	A VB app is viewed in these three states

To select more than one control: Shift+click, or drag a selection rectangle. Align with Format menu.

V2 — Event-driven idea and data types

Definition — Event-driven programming

The order of execution is decided by user or system events, not by a single top-to-bottom script. You write Command1_Click; VB calls it when the user clicks that button.

Type	Stores
Integer	Whole numbers in a limited range

Type	Stores
Long	Larger whole numbers
Single / Double	Fractional numbers
String	Text
Boolean	True / False
Variant	Can hold many kinds (default if you omit As)
Currency / Date	Money with 4 decimals; calendar dates

```

Private Sub Command1_Click()
    Dim number1 As Integer, number2 As Integer, number3 As Integer
    Dim total As Variant, average As Variant
    number1 = Val(Text1.Text)
    number2 = Val(Text2.Text)
    number3 = Val(Text3.Text)
    total = number1 + number2 + number3
    average = total / 3
    Label1.Caption = total
    Label2.Caption = average
End Sub

```

Three numbers → total and average (model-paper style)

V3 — Control structures

```

' If Then Else
If marks >= 40 Then
    Label1.Caption = "Pass"
Else
    Label1.Caption = "Fail"
End If

' odd numbers 1 to 19
Dim i As Integer
For i = 1 To 19 Step 2
    Print i
Next i

' Select Case similar to C switch
Select Case colour
    Case 1: Print "Red"
    Case 2: Print "Black"
    Case Else: Print "Other"
End Select

```

Relational: = <> < > <= >=. Logical: And Or Not Xor. Do While / Do Until / For Next / For Each are the loops.

V4 — ListBox, ComboBox, methods

ListBox shows many items; ComboBox is a drop-down that also allows typing (depending on Style). AddItem adds a row; RemoveItem index deletes; Clear empties; ListIndex is the selected item; Text / List(i) reads items.

V5 — Tiny VB projects for practicals

- Calculator: four command buttons, two text boxes, one label.
- Traffic of if/case: convert a number 1-3 into colour names.
- Login form: compare TextBox password with a constant; MsgBox.

- Counter: add 1 on each click; reset button.

Part C — Sindh Curriculum 2019 units

Unit 1 — System Development Life Cycle (weight 10%)

Definition — System

A set of related components that work together toward a goal (people + hardware + software + data + procedures).

Definition — SDLC

A structured sequence of stages used to plan, build, test, release and maintain software so that the product meets user needs and can be maintained.

Stakeholder	Phase they own
Project manager	Whole project: time, cost, people
Business analyst	Requirements
System architect	Design
Developers	Coding / implementation
Testers	Testing
Operations team	Deployment
Production support	Maintenance

Phases

- Planning: problem identification, scope, feasibility (technical, economic, operational, schedule).
- Requirement engineering: gather functional (what the system must do) and non-functional (speed, security, usability) needs. Write an SRS (Software Requirements Specification).
- Design: architecture, algorithms, flowcharts, database schema, UI mock-ups.
- Development / implementation: actual coding and unit tests.
- Testing: find bugs and errors; compare behaviour with the SRS.
- Deployment: install in the real environment; SLAs (Service Level Agreements) may define uptime.
- Maintenance: correct faults, adapt to new rules, improve performance.

Board exam tip

For 8 marks: define SDLC, list phases in order, write two sentences on each, name three stakeholders.

Unit 2 — Pointers in C++ (weight 15%)

Definition — Pointer

A variable that stores the memory address of another variable. Significance: dynamic memory, efficient array passing, data structures, and true pass-by-address.

```

#include <iostream>
using namespace std;
int main() {
    int n = 25;
    int *p;          // declaration: p will hold address of an int
    p = &n;          // reference operator & = 'address of'
    cout << "Address = " << p << endl;
    cout << "Value   = " << *p << endl;    // dereference *
    *p = 40;         // changes n
    cout << n;       // 40
    return 0;
}

```

Address and value of a variable using a pointer

```

void double_both(int *x, int *y) {
    *x = *x * 2;
    *y = *y * 2;
}
int main() {
    int a = 3, b = 5;
    double_both(&a, &b);    // send addresses
    cout << a << " " << b; // 6 10
}

```

Curriculum practical: double two values via pointer arguments

Common mistake

* in a declaration means 'pointer to'. * in an expression means 'value at that address'. & is only 'address of' (unless used in a C++ reference).

Unit 3 — Object-oriented programming using C++ (weight 25%)

Largest XII unit in the 2019 curriculum. Four pillars plus class syntax will carry an 8-mark question.

Principle	Meaning in one exam sentence
Encapsulation	Bind data and functions in a class; hide details behind a public interface
Data hiding	Make data members private so outsiders cannot corrupt them
Inheritance	A derived class reuses and extends a base class
Polymorphism	One name, many forms: overloading (compile time) and overriding (run time)
Abstraction	Show essential features, hide inner complexity

Definition — Class / object

A class is a user-defined type (blueprint). An object is a variable of that class (instance) occupying memory.

Access specifiers

private: only member functions. public: anywhere. protected: class + derived classes. Default for class members is private.

```

#include <iostream>
using namespace std;

class Student {
    int age;           // private by default
    float percentage;
public:
    Student() { age = 0; percentage = 0; }           // default constructor
    Student(int a, float p) { age = a; percentage = p; } // overloaded
    void input() { cin >> age >> percentage; }
    void show() { cout << age << " " << percentage << endl; }
    ~Student() { /* destructor: cleanup */ }
};

int main() {
    Student s1;           // default constructor
    Student s2(17, 85.5); // parameterised
    s1.input();
    s1.show();
    s2.show();
}

```

STUDENT class — curriculum practical 1–2

TIME class (practical)

```

class Time {
    int h, m, s;
public:
    Time(int hh = 0, int mm = 0, int ss = 0) : h(hh), m(mm), s(ss) {}
    void show() {
        cout << h << ":" << m << ":" << s;
    }
};

```

Inheritance and polymorphism

```

class Base {
protected:
    int x;
public:
    void set(int v) { x = v; }
    void show() { cout << "Base " << x << endl; }
};

class Derived : public Base {
    int y;
public:
    void set(int a, int b) { x = a; y = b; } // overload (different params)
    void show() { cout << "Derived " << x << ", " << y << endl; } // override
};

int main() {
    Derived d;
    d.set(2, 3);
    d.show();
}

```

Base / derived, overloading and overriding

Constructor: special member with the class name, no return type, called automatically when an object is born. Destructor: ~ClassName, called when the object dies. Constructor overloading = several constructors with different parameters.

Unit 4 — File handling (weight 15%)

Definition — File handling

Reading and writing data on secondary storage so it survives after the program ends. A stream is a sequence of bytes flowing between a program and a file.

Idea	Detail
Text file	Human-readable characters, e.g. .txt
Binary file	Raw bytes; can store complete objects
ios::in	Read (default for ifstream)
ios::out	Write, truncate existing
ios::app	Append
ios::binary	Binary mode
BOF / EOF	Beginning / end of file
get / put	Single character I/O
<< >> / getline	String / formatted I/O
read / write	Binary block I/O

```

#include <fstream>
#include <iostream>
using namespace std;
int main() {
    ofstream out("notes.txt");
    out << "BIEK Computer Science\n";
    out.close();

    ifstream in("notes.txt");
    string line;
    while (getline(in, line))
        cout << line << endl;
    in.close();
}

```

Write and read a text file

```

struct Student { char name[20]; int age; float perc; };
Student s = {"Ali", 17, 88};
ofstream f("stu.dat", ios::binary);
f.write((char*)&s, sizeof(s));
f.close();
ifstream g("stu.dat", ios::binary);
g.read((char*)&s, sizeof(s));
cout << s.name << " " << s.age << " " << s.perc;

```

Binary file of a student object

Unit 5 — Database fundamentals (weight 15%)

Definition — Database

An organised collection of related data. A DBMS (Database Management System) is software that defines, stores, protects and retrieves that data (examples: MS Access, MySQL, Oracle).

Flat file	DBMS
One table in a spreadsheet or text file	Many related tables
Redundant data, hard to query	Less redundancy, SQL queries

Flat file	DBMS
No multi-user control	Concurrency, backup, security

Applications: library, hospital, bank, airline, college admission, inventory. DBMS components: hardware, software, data, procedures, people, data access language (SQL). Characteristics: accuracy, integrity, controlled redundancy, security, scalability, normalisation.

Users

DBA (admin, backup, accounts). Application programmer. Naive / end user (forms and reports).

Data abstraction

- Physical level — how bits sit on disk (indexes, files).
- Logical / conceptual — tables, columns, relationships.
- External / view — what a particular user is allowed to see.

Relational terms

Term	Meaning
Table / relation	A 2-D grid of rows and columns
Tuple / record	One row
Attribute / field	One column
Domain	Allowed values of an attribute
Schema	Design of the database (structure, not the data)
Primary key	Uniquely identifies a row; NOT NULL
Foreign key	Attribute that refers to a primary key in another table

ER modelling

Entity: a real-world object (STUDENT, BOOK). Attribute: property (RollNo, Title). Relationship: association. Cardinality: 1:1, 1:N, N:1, M:N. Diagram: rectangles = entities, ovals = attributes, diamonds = relationships, underlines = keys.

Normalisation

Form	Rule (exam wording)
1NF	Atomic values; no repeating groups
2NF	1NF + every non-key attribute depends on the whole primary key
3NF	2NF + no non-key attribute depends on another non-key (no transitive dependency)

SQL

```

CREATE TABLE Book (
    BookId INT PRIMARY KEY,
    Title VARCHAR(80) NOT NULL,
    Author VARCHAR(50),
    Copies INT
);
INSERT INTO Book VALUES (1, 'C Programming', 'Kernighan', 5);
SELECT Title, Author FROM Book;           -- projection
SELECT * FROM Book WHERE Copies > 2;      -- selection
UPDATE Book SET Copies = Copies + 1 WHERE BookId = 1;
DELETE FROM Book WHERE BookId = 1;

-- operators: arithmetic + - * /   comparison = <> < >   logical AND OR NOT
-- equijoin example
SELECT Student.Name, Book.Title
FROM Student, Issue
WHERE Student.RollNo = Issue.RollNo;      -- join on equality
-- cross join: every row of A with every row of B (Cartesian product)

```

DDL/DML the curriculum names: CREATE, INSERT, SELECT, UPDATE/DELETE

Selection picks rows (WHERE). Projection picks columns (the list after SELECT). Library database planning: purpose → tables (Book, Member, Issue) → fields → unique keys → relationships up to 3NF → refine.

Unit 6 — Introduction to multimedia (weight 15%)

Definition — Multimedia

The combined use of more than one medium — text, graphics, animation, audio and video — to communicate. Importance: clearer teaching, marketing, entertainment, simulations, accessibility (captions + audio).

Component	Typical file types
Text	.txt .doc .pdf
Graphics (still)	.jpg .png .gif .bmp .svg
Animation	.gif .mp4 (plus project files of an editor)
Audio	.mp3 .wav .aac
Video	.mp4 .avi .mkv .mov

Packages: web apps (YouTube editor, Canva, CapCut web) vs desktop (Audacity, Shotcut, OpenShot, DaVinci; older lists mentioned ProShow Gold, JetAudio). Scene/canvas is the working stage; layers stack pictures/text; keyframes mark a property at a time so the computer interpolates motion.

- Practical: 2-minute video using text, stills, voice and a soundtrack.
- Export MP4 (wide compatibility) and at least one other format.
- Upload/share: YouTube unlisted, Google Drive, school LMS — respect copyright of music.

Unit 7 — Wireless and mobile communication (weight 10%)

Definition — Wireless communication

Transfer of information without a physical guided medium, using electromagnetic waves.

Definition — Mobile communication

Wireless communication with users who move, typically cellular phones handed from cell to cell.

Term	Meaning
Frequency spectrum	Range of usable radio frequencies, regulated by the state
Radio signal	EM wave carrying the message
Transceiver	Transmitter + receiver in one unit (a phone)
Access point	Wi-Fi device that connects clients to a LAN/Internet
Line of sight	A clear straight path; needed by many microwave links

Short-distance technologies

Tech	Typical range	Notes
Infrared	A few metres, line of sight	Old phone beaming, TV remotes
Bluetooth	~10 m (class 2)	Earbuds, file share
Wi-Fi	Tens of metres	IEEE 802.11 LAN
Microwave	km with dishes	Backhaul links
WiMAX	km (city)	Wireless MAN idea

Satellite orbits (long distance / GPS family)

Orbit	Height idea	Use
GEO	~36 000 km, appears fixed	TV broadcast, some weather
MEO	Thousands of km	Classic GPS constellation
LEO	Hundreds of km	Phones, Earth imaging, new internet constellations

Cellular network

A city is divided into cells, each with a Base Transceiver Station (BTS). A Base Station / BSC plus a Mobile Switching Centre (MSC) connect cells to the telephone/Internet core. Frequencies are reused in non-adjacent cells to save spectrum.

Generation	Headline technology
2G	GSM digital voice, SMS
2.5/3G	GPRS/UMTS — mobile data
4G	LTE — fast IP data, HD video
5G	Higher capacity, lower latency (curriculum: 'future' — now deployed in parts of Pakistan)

Definition — Ad hoc network

A network without a pre-built infrastructure (no fixed AP). A mobile ad hoc network (MANET) is one whose nodes move. A fixed ad hoc net stays in one place (two laptops cabled or Wi-Fi direct on a desk).

Part D — Grade XII practicals

- 1 Class STUDENT with age and percentage; input/show via object.
- 2 Initialise objects with constructors; TIME class with h:m:s.
- 3 Inherit DERIVED from BASE; demonstrate overloading and overriding.
- 4 Print address and value with a pointer; double two variables via pointer arguments.
- 5 Text file write/read; binary file of a student object.
- 6 SQL: CREATE TABLE, ALTER (add/drop column), DROP TABLE, INSERT/UPDATE/DELETE, SELECT with WHERE and operators.
- 7 Multimedia: 2D/3D text, import image, layers, keyframes, 2-minute video, export, upload.

Model practice — Option I (C)

Section A style MCQs

1. C language was developed by
 - (A) Bill Gates
 - (B) Dennis Ritchie ✓
 - (C) Bjarne Stroustrup
 - (D) James Gosling
2. Every C statement ends with
 - (A) .
 - (B) ,
 - (C) ; ✓
 - (D) :
3. Preprocessor directives begin with
 - (A) &
 - (B) //
 - (C) # ✓
 - (D) <
4. Execution of a C program starts at
 - (A) start()
 - (B) begin()
 - (C) main() ✓
 - (D) printf()
5. A character constant is enclosed in
 - (A) double quotes
 - (B) single quotes ✓
 - (C) braces
 - (D) parentheses
6. Which is an escape sequence?
 - (A) "/n"
 - (B) "\\t" ✓
 - (C) "%f"
 - (D) i++
7. 4 % 7 equals
 - (A) 0
 - (B) 1
 - (C) 4 ✓
 - (D) 5
8. Logical operators are
 - (A) <= >=

- (B) && || ✓
- (C) += =
- (D) / %

9. Which is NOT a loop in C?

- (A) do-while
- (B) while
- (C) repeat-until ✓
- (D) for

10. Loop that runs at least once

- (A) for
- (B) do-while ✓
- (C) while
- (D) goto

11. continue cannot be used with

- (A) for
- (B) while
- (C) do-while
- (D) switch ✓

12. switch cannot directly test

- (A) char
- (B) int
- (C) float ✓
- (D) none

13. Keyword to send a value back from a function

- (A) switch
- (B) goto
- (C) go back
- (D) return ✓

14. Array elements are stored

- (A) at random gaps
- (B) in consecutive memory ✓
- (C) only on disk
- (D) in ROM

15. printf can print

- (A) char
- (B) string
- (C) numbers
- (D) all of these ✓

Section B samples

- 1 What is an IDE? Which two Turbo C++ keys compile and run?
- 2 Define header files. Why do we write #include <stdio.h>?
- 3 Classify: record1, \$tax, 0.576, 'a', "Pak", 27,822.
- 4 Escape sequence versus format specifier.
- 5 Five basic data types with Turbo C sizes.
- 6 Local versus global variables with a tiny example.
- 7 Syntax of prototype, call and definition.
- 8 Declare and initialise an array two ways.

Section C samples

- 1 What are programming languages? Explain syntax, runtime and logical errors with examples.
- 2 Explain for, while and do-while with syntax and a small example of each.
- 3 Write a C program to print the 666666...1 digit pattern OR to sort ten integers.

- 4 Convert a colour if-else ladder into switch-case (Red/Black/White/No colour).

Model practice — new-book long questions

- 1 Define SDLC. Explain its phases and name four stakeholders.
- 2 What is a pointer? Write a program that prints address and value and that doubles two numbers via pointers.
- 3 Explain encapsulation, inheritance and polymorphism with a short C++ sketch.
- 4 Differentiate text and binary files. Write programs to write/read both.
- 5 What is a DBMS? Draw an ER diagram of a library and write SQL to create and query Book.
- 6 Define multimedia. Describe its five components and the idea of layers and keyframes.
- 7 Explain cells, BTS and MSC. Compare 2G, 3G, 4G and 5G in a table.

Quick glossary (XII)

Term	One-line meaning
Actual parameter	Value passed at the call site
Constructor	Class member that initialises a new object
Encapsulation	Data + functions wrapped in a class
EOF	End of file condition
Formal parameter	Variable in the function definition
Inheritance	Derived class reuses a base class
IDE	Editor + compiler in one tool
MANET	Mobile ad hoc network
MSC	Mobile Switching Centre
Normalisation	Table design that reduces redundancy
Pointer	Variable that stores an address
Polymorphism	One interface, many implementations
SDLC	Stages of building and maintaining software
SQL	Language to define and query relational data
Stream	Sequence of bytes in or out of a program

End of Class XII lectures. Revise Part A or B according to your option, then use Part C for 2019-curriculum colleges. Run every program once before the practical exam.